

CURRENT CODEBASE RUNDOWN

3DG Engine

Architecture, Structure, and Systems

A developer-facing map of the runtime library, editor, asset pipeline, gameplay framework, standalone player, and the systems that connect them.

113

runtime public headers

54

editor public headers

19

regression suites

Codebase snapshot: 9 August 2026

C++17 | OpenGL | ECS | Native assets | C++ and Lua scripting

DOCUMENT MAP

How to use this guide

This is the architectural rundown: it explains ownership, boundaries, data flow, execution order, and the purpose of each major system. It is intentionally shorter than the full API-oriented engine manual.

Primary rule

Runtime capability belongs in engine. Project behavior belongs in game or project scripts. Authoring-only behavior belongs in editor. The standalone player loads exported runtime data and must not depend on editor code.

#	Section	Focus
01	Architecture at a glance	Layers, dependencies, ownership
02	Repository and build structure	Targets, folders, third-party foundations
03	Application and frame lifecycle	Variable update, fixed update, render
04	ECS, scenes, prefabs, and streaming	Runtime world organization
05	Asset architecture	Identity, registry, import, load, cook
06	Rendering stack	PBR, shadows, environment, post process
07	World rendering	Terrain, foliage, grass, water, splines
08	Physics and collision	Bodies, filters, queries, controller, ragdoll
09	Animation and characters	Skeletons, graphs, actions, sockets
10	AI and navigation	NavMesh, agents, BT, blackboards
11	Gameplay and scripting	C++, Lua, timers, events, modules
12	Audio, particles, UI, and cameras	Presentation and player-facing systems
13	Editor architecture and tools	Panels, controllers, asset editors
14	Diagnostics and optimization	Profiler, traces, validation, tests
15	Extension patterns	Adding systems, assets, and graphics APIs
16	Reference appendices	Formats, source map, glossary

01 - SYSTEM BOUNDARIES

Architecture at a glance

3DG Engine is organized as a reusable runtime with separate authoring and game layers. Dependencies point inward toward the engine; exported data crosses from the editor into the player.

Target dependency map

Target	Depends on	Responsibility
engine	GLFW, GLAD, GLM, OpenGL, Assimp, ImGui, miniaudio, Lua	Reusable runtime systems and public APIs. Never depends on game or editor code.
game	engine	Engine-shipped/manual project scripts and shared registration.
game_scripts	engine	Hot-reloadable project-owned C++ scripts generated from Content/Scripts.
3DGEEditor	engine, MaterialMaker, game	Scene and asset authoring, Play mode, import, validation, cooking.
3DGScriptCompiler	project/build configuration	Editor-managed C++ script compilation helper.
3DGLauncher	engine	Project discovery, creation, selection, and launch of the matching editor.
player	engine, game	Standalone runtime for cooked/exported project content.
tests	focused engine/editor units	Nineteen subsystem regression executables registered with CTest.

Data moves across boundaries



- Import converts external FBX, glTF, OBJ, PNG, JPEG, and related files into engine-owned native assets.
- The editor stores stable asset IDs plus readable fallback paths in authored documents.
- Editor Play exports a runtime representation of the current scene; Stop restores the authored state.
- The cooker walks dependencies and stages only runtime-reachable content for the player.

Ownership invariant

GPU wrappers and runtime resources are owned by caches or system objects. ECS components generally hold lightweight state or non-owning references. Clear entities before destroying the caches they reference.

02 - SOURCE LAYOUT

Repository and build structure

```
3DGEEngine/  
engine/include/engine/ public runtime API (113 headers)  
engine/src/ runtime implementations (93 sources)  
editor/include/ editor API and panel declarations (54 headers)  
editor/src/ app, controllers, panels, assets, launcher (57 sources)  
game/ shared game module and script export bridge  
player/ standalone runtime host  
MaterialMaker/ material authoring library  
tests/ 19 regression executables  
Content/ default project content and scripts  
docs/ manuals and task-focused tutorials  
output/pdf/ generated PDF documentation
```

Build foundations

Dependency	Role
GLFW	Window creation, OpenGL context, input, event pump.
GLAD + OpenGL	Current graphics backend and function loading.
GLM	Vectors, matrices, quaternions, transforms.
Assimp	FBX, glTF, OBJ, Collada, PLY, and STL import paths.
Dear ImGui docking	Editor shell, panels, inspectors, graphs, tools.
miniaudio	Audio device, voices, spatial playback, mixing foundation.
Lua 5.4	Embedded runtime for project gameplay scripting.

Build and packaging

CMake 3.16+ builds C++17 targets into a common binary directory. On Windows the editor, launcher, and helper are staged under the editor runtime folder. The player has install rules for a relocatable folder and CPack ZIP packaging. A cooked project contains Content, player.cfg, AssetRegistry.3dgdb, and CookManifest.3dgmanifest.

Clean boundary

If a feature is required by both Editor Play and a packaged game, its implementation belongs in engine. The editor should author its data, not own the runtime behavior.

03 - RUNTIME ORDER

Application and frame lifecycle

engine::Application provides the common loop. A frame is capped after long stalls, variable work runs once, fixed simulation consumes an accumulator, and render interpolation uses the remaining fraction.



Phase	Typical work	Clock
OnInit	Create GL resources, load runtime assets, build systems and caches.	Once, after context exists
OnUpdate	Input intent, scripts, animation blend, audio, camera, streaming, HUD.	Frame delta
OnFixedUpdate	Controllers, AI movement, physics, projectiles, contacts, ragdoll.	Fixed delta, default 1/120 s
OnRender	Shadows, scene geometry, environment, particles, post process, HUD, debug.	Once per visible frame
Window::Update	Swap buffers, pump events, reset per-frame input deltas.	End of frame
OnShutdown	Destroy live instances before their factories, modules, or caches.	Once

Time ownership

Gameplay uses scaled time. Global time dilation affects movement, AI, animation, particles, cameras, timers, and physics. Hit-stop uses an unscaled release clock so a full freeze can end. Editor UI and audio mixing remain responsive on unscaled time.

Interpolation and stability

- Never apply the same movement in both variable and fixed updates.
- Use fixed-step sweeps for collision-sensitive projectiles and controllers.
- Use interpolation only for presentation; authoritative state remains on fixed snapshots.
- Resource creation that needs OpenGL must wait until initialization or a lazy panel draw.

04 - WORLD ORGANIZATION

ECS, scenes, prefabs, and streaming

The registry owns entities and typed component stores. Systems iterate component combinations; editor objects serialize into runtime components rather than carrying editor panel state into the player.

Core data model

Concept	Purpose	Examples
Entity	Lightweight identity used to address component records.	Scene object, light, character, emitter, empty system host
Transform	Authoritative scene position, rotation, and scale.	Independent from model import/render correction
Render components	Static/skinned model, material, light, terrain, water, foliage.	Runtime cache-backed resources
Behavior components	Health, scripts, player controller, AI agent, audio, particle state.	Updated by focused systems
Physics components	RigidBody and Collider authored state.	Converted to PhysicsWorld bodies/shapes
Prefab	Reusable object/component hierarchy.	Palette placement and runtime instantiation

Scene lifecycle



- Scenes normally live under Content/Scenes; Save overwrites the active asset and Save As creates a new one.
- The last successfully saved scene is recorded per project and restored on reopen.
- RuntimeSceneLoader resolves components and resources without depending on editor headers.
- LevelStreamingManager activates and unloads level assets around a viewer while preserving persistent content.

Level-as-asset streaming

A .3dgworld manifest references a persistent level and streamed scene levels. World-space placement is applied during load, activation is policy-driven, and unload releases level-owned entities. This separates large-world composition from the contents of each scene asset.

Transform separation

Asset-level model correction fixes imported axes, scale, and facing. Scene Transform remains the gameplay/world transform used by gizmos, physics, navigation, and scripts.

05 - IDENTITY, IMPORT, LOAD, COOK

Asset architecture

The asset system converts external sources into engine-owned payloads, assigns stable 128-bit handles, tracks dependencies, and resolves them through AssetRegistry.3dadb.



Asset family	Formats	Notes
Native geometry	.3dgmesh, .3dgskmesh	Static and skeletal meshes with engine-owned payloads.
Animation	.3dgskel, .3dganim	Skeleton can be selected/reused; animation import can omit mesh and skeleton.
Textures	.3dgtex	Engine texture metadata/payload, including 16-bit PNG decode path.
World data	.3dgworld, .3dgfoliage	Streaming manifest and instanced foliage palette.
Authored visual	.3dgmat, .3dgshader, .particle, .particlefx	Material, typed shader graph, emitter, multi-emitter effect.
Gameplay/editor	.3dgcharacter, .3dgclip, .3dggraph, .btgraph, .hud	Character, action clip, animation graph, behavior tree, HUD.
Scene/audio	.scene, .3dgaudio, .3dgmixer, .3dgmusic	Level documents and authored audio assets.

Import and reimport

- The native file dialog selects a system source; the import dialog selects or creates a project destination folder.
- Asset-specific settings open during import. Static, skeletal, animation, texture, audio, and terrain paths retain importer metadata.
- Skeletal and animation imports can reuse an existing skeleton from a registry-backed dropdown.
- Reimport uses the remembered source and settings while preserving the stable asset handle.

Loading and cooking

RuntimeAssetManager resolves IDs, validates expected types, caches models/textures/materials/shaders, and keeps readable paths as recovery fallbacks. AssetCooker starts from a scene or world, follows registry dependencies, stages reachable payloads and Lua scripts, writes a cook manifest, and rejects incomplete packages.

Identity rule

Renaming or moving an engine-owned asset does not change its AssetHandle. References should prefer the ID and treat the path as a human-readable fallback.

06 - GRAPHICS AND LIGHTING

Rendering stack

The current backend is OpenGL 3.3 core. Rendering is split into resource wrappers, scene submission, specialized geometry paths, lighting/shadows, environment, and screen-space/post-processing stages.

Frame rendering pipeline



Subsystem	Responsibility
Renderer / Mesh / Model	Static geometry submission, buffers, vertex layouts, draw execution.
SkinnedRenderer / SkinnedModel	Bone palettes, skinned meshes, animated model drawing.
PbrRenderer	Main material/lighting path for registry scene objects.
Lighting	Directional, point, and spot lights; clustered-light support.
Shadows	Cascaded directional, point cubemap, spot, caster collection, visibility distance.
Environment	Skybox, procedural sky, day/night, clouds, IBL.
Screen-space	SSAO and SSR with runtime feature controls.
Post process	Render-scale output and ordered shader/effect stack.
Debug lines	Gizmos, colliders, traces, sockets, nav and editor guides without cube helpers.

Materials and shaders

MaterialAssetLoader resolves PBR scalar values, texture maps, blend/surface settings, and optional custom shader assets. RuntimeShaderManager compiles and caches shader variants. ShaderGraphCompiler translates typed graph nodes to executable shader source with domain-specific outputs and parameters.

Performance model

- Frustum culling and visibility tests prevent submission of off-screen objects.
- LOD, batching/instancing, cached asset resources, and reduced editor overlay work lower CPU and draw-call cost.
- GpuProfiler separates scene GPU time from UI and reports frame metrics alongside CPU submission/build costs.
- Render-scale and optional SSAO, SSR, cloud, shadow, and post-process features provide quality controls.

07 - WORLD RENDERING

Terrain, foliage, grass, water, and splines

System	Authoring model	Runtime behavior
Terrain	Heightmap/Gaea import, sculpt brushes, cursor, material and layer settings.	Chunked/optimized geometry, culling, shading, collision/navigation input.
Grass	Density, scale, wind, placement, toggled height randomizer.	Instanced blades with randomized height and distance controls.
Foliage	Foliage asset palettes, paint/scatter, per-type density and rules.	Instanced static meshes, culling/LOD, optional collision proxies.
Water	Lake or river settings, reflection/refraction, transparency, flow.	Surface rendering, underwater state, interaction and camera response.
Spline	Editable points, tangents/orientation, per-point rotation, script access.	Sampled paths for rivers, camera rails, placement, and gameplay.

Terrain

- Sculpting updates only affected regions and uses throttling/dirty ranges to keep 1024-resolution editing responsive.
- The viewport projects a circular brush indicator onto the terrain before an edit.
- Material blending, normals, camera clipping/culling, and terrain bounds are handled as terrain-specific concerns.

River spline contract

A river surface is generated along its spline centerline. Width frames follow the sampled tangent, point rotations control local banking/twist, and UV/flow coordinates advance by accumulated arc length so flow remains continuous through bends. Moving the river object moves the spline and generated plane together.

Foliage performance

- Palette assets separate reusable foliage types from scene instance data.
- Instancing amortizes draw calls; spatial partitions and distance culling reduce visible instance work.
- Collision is represented by limited proxies rather than per-instance full mesh physics.

Authoring/runtime split

Brush cursors, spline handles, paint previews, and placement helpers are editor overlays. Generated terrain, water, foliage, and collision data are the runtime payloads.

08 - SIMULATION AND QUERIES

Physics and collision

PhysicsWorld consumes ECS RigidBody and Collider data, performs fixed-step integration and collision, supports filtered queries, and writes authoritative results back to scene transforms.

Area	Capabilities
Bodies	Static, dynamic, and trigger/overlap behavior; velocity, gravity, material properties.
Shapes	Box, sphere, capsule, true cylinder, cone, plane, and primitive-specific colliders.
Filtering	Object channels, presets, per-channel Block/Overlap/Ignore, owner ignores.
Queries	Line, sphere, and shape traces with hit data and debug drawing.
Joints	Distance, rope, and spring relationships.
CharacterController	Upright controller capsule, grounding, slopes, steps, gravity, smooth stair handling.
Ragdoll	Animated-to-physical body transition on death and pose synchronization.

Controller capsule

- Radius and total height are independent; total height includes both rounded caps and is clamped to at least two radii.
- Character asset preview values are converted consistently to scene/controller units.
- The controller capsule, not an unrelated primitive collider, owns character movement collision.
- Sprint is disabled while airborne; grounded, falling, speed, and movement flags feed animation logic.

Collision policy examples

Pair	Recommended response
Player vs WorldStatic	Block
Player vs Collectible	Overlap
Camera probe vs Collectible	Ignore
Projectile vs owner	Ignore at spawn; clear only when safe
Projectile vs target	Block or overlap, then apply damage from confirmed hit

09 - MOTION ARCHITECTURE

Animation and characters

Animation assets are separated into engine-owned skeleton/clip data and authored graphs/action clips/character definitions. A character asset gathers the model, collider, movement, camera, animation, sockets, scripts, AI, and gameplay defaults.

Animation data flow



System	Purpose
Skeleton	Named hierarchy and bind/inverse-bind data shared across compatible assets.
Animator	Samples clips and produces bone matrices.
Animation graph	States, parameters, blend spaces, multi-condition transitions, locomotion.
Blend space	Smoothly blends idle/walk/run samples from speed or multidimensional parameters.
Action clip	Standalone script-callable attack/cast/hit animation with events and playback rules.
Layering	Upper-body or masked actions can play while locomotion continues.
Montage-style lock	A non-layered action can block character movement until completion.
Root motion	Optional extraction and controlled application to scene/controller movement.
Sockets	Named bone-relative attachment/spawn transforms, editable and visible in Character Editor.

Locomotion responsibilities

- The controller and physics produce speed, grounded, falling, direction, and vertical velocity state.
- The animation graph reads those values; the behavior tree does not choose locomotion clips.
- Blend response and rotation interpolation smooth state changes and direction changes.
- Action scripts request saved action clips by asset/name and use events for hit or projectile timing.

Character Editor

The editor previews the selected skeletal model and animation graph, shows collider boundaries, manages materials, sockets and staff attachments, supports multiple scripts, and exposes AI/team and camera settings. Preview/model correction transforms are asset-level; placed scene transforms remain independently editable.

10 - DECISION AND MOVEMENT

AI and navigation

AI is deliberately split: perception and behavior decide intent, navigation finds a route, and AI movement/controller code performs grounded motion. Animation observes the resulting movement state.



Subsystem	Responsibility
NavMeshBuilder	Build traversable polygons from geometry inside a NavMesh Bounds Volume.
NavMesh / AStar	Store walkable connectivity and find paths.
AiAgent	Speed, force, reach radius, repath interval, vision, team, target, patrol.
AiMovement	Horizontal path following plus gravity, floor probes, slope and stair constraints.
Perception	Vision range/cone, focus target, sight loss, team-aware hostile selection.
Behavior tree	Selectors/sequences, tasks, decorators, services, blackboard-driven decisions.
BT scripts	Project-defined tasks integrated into graph authoring and runtime registration.

Behavior tree architecture

- Blackboard keys are typed shared state for targets, ranges, flags, positions, cooldowns, and game state.
- Tasks perform bounded actions such as Focus Target, Clear Focus, Chase, Patrol, Attack, or Script Task.
- Decorators gate a branch; boolean checks use true/false values and may combine multiple conditions.
- Services refresh perception or blackboard facts while their branch is active.
- Composite display names improve graph readability without changing node semantics.

Team and stopping behavior

Non-zero teams identify hostiles; auto-targeting selects a visible agent on a different team. Chase stops inside reach/attack range, resumes when the target leaves range, and updates facing continuously. Navigation remains horizontal while the controller handles stairs and ground height.

Debug meaning

Red dashed navigation markers are debug visualization, not the NavMesh itself. Play-mode debug lines should appear only when the matching gameplay debug toggle is enabled.

11 - PROJECT LOGIC

Gameplay and scripting

Gameplay code can be authored as native C++ scripts or Lua scripts. Both execute in Editor Play and packaged builds through the same component/lifecycle model and script-facing engine services.

Layer	Role
Script base/API	Lifecycle callbacks, entity/component access, spawning, queries, timers, audio, particles, HUD, animation, splines, debug traces.
ScriptComponent	One or more attached script slots with class, language, fields, enabled state, and instance.
Script factory	Maps class names to native constructors; generated registration keeps editor-created scripts discoverable.
game static module	Engine-shipped/manual scripts linked by editor and player.
game_scripts DLL	Project-owned C++ scripts compiled and hot-reloaded by the editor.
Lua VM	Embedded Lua 5.4 runtime with matching engine bindings and project Content scripts.

Lifecycle and order

```
OnCreate / OnEnable
-> OnUpdate(dt) variable gameplay update
-> OnFixedUpdate(dt) fixed simulation update
-> collision, trigger, animation-event, and gameplay callbacks
OnDisable / OnDestroy
```

Shared services

- Named timers can call an event or function after a delay, repeat, pause, resume, and clear by handle/name.
- Global time dilation supports slow motion; hit stop temporarily applies a stronger dilation and restores safely.
- Event and named-value services decouple gameplay logic from HUD, objectives, and other systems.
- Gameplay debug functions draw line, sphere, capsule, box, and trace results during Play when enabled.
- Safe entity handles and validation protect scripts from accessing destroyed runtime objects.

Editor workflow

Scripts are created under Content/Scripts from templates, including AI/BT templates. The editor validates the class name, updates project-generated includes/registration, compiles through 3DGScriptCompiler, exposes saved scripts in dropdowns, and opens the chosen external code editor. C++ and Lua assets are included by cooking so the same logic reaches the build.

12 - PRESENTATION SYSTEMS

Audio, particles, UI, and cameras

Audio

Asset/runtime concept	Capabilities
Audio source	Clip, volume, pitch, loop, spatial settings, attenuation, autoplay.
Cue	Variation, randomization, layers, routing, and gameplay-triggered playback.
Mixer	Buses, gains, effects, snapshots, and monitoring.
Music	Adaptive sections/states and transitions.
RuntimeAudioSystem	Synchronizes ECS sources and listener state with AudioEngine.
Audio editor	Asset creation, preview, cue/music authoring, mixer setup.

Particles

Particle assets compile an ordered module stack into CPU or GPU emitter pipelines. Spawn modules define rate, bursts, shape, lifetime, and initial velocity; update modules apply forces, rotation, color/size over life, drag, and collision; render modules define sprite/billboard, texture, blend, and trails. A `.particlefx` combines multiple emitters into one effect.

HUD and UI

- Runtime HUD assets contain Panel, Text, Bar, Button, and Image widgets with anchors and pixel offsets.
- Named float/string bindings update health, score, time, game state, mana, and custom script values.
- Buttons emit named actions consumed by gameplay/menu logic.
- World-space enemy health bars project entity health into screen space.

Camera and cinematics

- Game mode selects the gameplay camera once before Play; gameplay input does not arbitrarily toggle modes.
- Modes include free/third-person and fixed-angle isometric behavior, with collision probing and recovery.
- Camera Manager stores cameras, blends, zones, shakes, rails, sequences, and cinematic cues.
- Camera sequences use splines/keyframes and can coordinate gameplay events and letterbox/HUD state.

13 - AUTHORIZING HOST

Editor architecture and level-design tools

EditorApp coordinates project state, dockspace, scene, viewport, renderer, Play mode, content, panels, and platform dialogs. Responsibilities are split into controllers and specialized panels rather than kept in one application file.

Editor layer	Main responsibility
EditorProject	Project paths, settings, Content root, last scene, project metadata.
EditorScene	Editable objects, component authoring, save/load, selection state.
EditorRuntimeController	Export current scene, start/stop Play, restore authoring state.
EditorContentController	Browse, classify, import, reimport, rename, copy/cut/paste/delete, open assets.
Viewport + camera/mouse	Picking, capture, navigation, framing, viewport bounds, input ownership.
Transform/Gizmo	W move, E rotate, R scale, G uniform scale, world/local, snapping.
EditorLineRenderer	Selection outlines, colliders, sockets, traces, navigation, splines, helpers.
Dockspace/Panels	Grouped, dockable tools with mouse-capture protection for panel fields.

Asset editors

Double-click routing opens the correct panel for materials, shaders, particles, HUDs, characters, animation graphs, action clips, behavior trees, prefabs, audio, meshes, scenes, worlds, and foliage. Registry-backed dropdowns replace manual path typing wherever practical.

Level-design toolset

Tool group	Tools
Blockout	Blockout primitives, Room Builder, staircase and architectural primitives.
Placement	Modular Placement, Prefab Palette, Array Tool, Scatter Paint, Alignment.
Organization	Level Layers, Level Variants, grouping, hierarchy, folders, naming.
Navigation	Viewport Bookmarks, camera framing, measurement, spline builder.
Validation	Level Validation, missing asset/script checks, export readiness.
Asset adjustment	Mesh Editor for pivot/origin, asset transforms, collision and bounds inspection.

Launcher

3DGLauncher is built beside the editor. It lists recent projects, creates a new project structure, opens an existing project, validates paths, and launches the matching editor executable without hard-coded installation paths.

14 - ENGINEERING SUPPORT

Diagnostics, optimization, validation, and testing

Facility	What it reports or controls
Profiler	FPS/frame graph, 1% low, CPU render, scene submit, UI build, GPU scene/UI/total, draw calls, object/triangle/vertex counts, lights, terrain, water, foliage, particles, VRAM.
Gameplay debugger	Runtime line/sphere/capsule/box traces, hit results, AI focus/path/vision, physics contacts and colliders.
Console	Asset, shader, script, physics, AI, import, cook, and runtime messages.
Script debugger	Attached instances, callbacks, compile/runtime state, errors, break/inspection support.
Level validation	Missing dependencies, bad references, unresolved scripts, malformed authored assets.
GPU profiler	Separates GPU scene and UI costs so CPU stalls are not misdiagnosed as GPU stalls.

Optimization architecture

- Visibility and frustum culling reduce scene submission; LOD and instancing reduce geometry and draw-call cost.
- Asset caches avoid repeat decoding/loading; dependency-pruned cooking reduces package and startup work.
- Terrain updates use regional dirty data; foliage uses instancing/spatial culling; particle backends select CPU/GPU appropriately.
- Fixed-step caps prevent spiral-of-death stalls; expensive editor scans/previews are throttled or cached.
- Optional render features and render scale allow quality/performance tradeoffs per project.

Regression coverage

Coverage group	Focused suites
Assets	registry, cooker, static mesh, skeletal asset, editor assets, foliage asset
Visual	shader asset, material/shader integration, image decode, terrain, particles
Gameplay	script system, project script tools, runtime AI, animation movement
World	world streaming, spline, camera manager, audio system

Serialization rule

Any change to shared formats, asset identity, scene records, or runtime ownership should run the focused suite first and then all 19 CTest regressions.

15 - GROWING THE ENGINE SAFELY

Extension patterns

Adding a runtime system

Step	Required decision
1	Define authoritative runtime data and who owns it.
2	Place reusable behavior in engine; keep authoring code in editor.
3	Choose update phase: variable, fixed, render, or event-driven.
4	Define serialization/versioning and failure behavior.
5	Add component/system integration and clear teardown order.
6	Expose a narrow script API, diagnostics, and focused tests.

Adding an asset type

- Register the type and extension; define stable identity, version, payload, and dependency extraction.
- Implement import/reimport when source data exists, plus registry scan and runtime loading/caching.
- Add Content-browser classification, icon, right-click operations, and double-click routing.
- Add validation, cooking, runtime fallback behavior, tests, and documentation.

Adding Vulkan or DirectX later

The present renderer exposes OpenGL concepts directly in several wrappers, so a second backend should begin with an explicit Render Hardware Interface rather than parallel conditionals throughout the engine.

RHI layer	Backend-neutral contract
Device	Initialization, capability queries, resource lifetime, frame synchronization.
Resources	Buffers, textures, samplers, render targets, pipelines, descriptor/binding sets.
Command context	Pass begin/end, barriers, draw/dispatch, copies, timestamps.
Shader toolchain	One source/intermediate strategy, reflection, binding layout, variant cache.
Presentation	Swap chain, resize, VSync/fullscreen, frame acquire/present.

Migration sequence

First isolate OpenGL behind the RHI while preserving behavior. Then add Vulkan or DirectX as a second implementation, port one render pass at a time, and compare both with the same scene/asset tests.

16 - QUICK LOOKUP

Reference appendices

Source ownership map

Area	Runtime source	Editor source
Core	engine/core	editor/app, editor/launcher
Scenes	engine/scene, engine/ecs	editor/scene
Assets	engine/assets	editor/assets, content controller
Graphics	engine/graphics	viewport, line renderer, visual panels
Physics	engine/physics	inspector, debug overlays, character editor
Animation	engine/animation	graph, clip, character editors
AI	engine/ai	behavior graph, BT script tools
Gameplay	engine/gameplay, game, player	script tools, runtime controller
Audio/UI	engine/audio, engine/ui	audio and HUD panels

Runtime checklist

- Load project configuration and resolve executable/project Content paths.
- Load AssetRegistry and runtime caches before scene components request assets.
- Load startup scene or .3dgworld persistent level, then initialize streaming.
- Register C++ module scripts and initialize the Lua environment before enabling script components.
- Create physics, AI, animation, audio, particle, camera, HUD, and rendering services in dependency order.
- Destroy live script/scene instances before unloading their modules or caches.

Key terms

Term	Meaning
Authored asset	Editable document created by an editor panel, such as a material, graph, HUD, or character.
Native asset	Engine-owned container produced by import, with a stable handle and versioned payload.
Runtime scene	Player-readable exported scene without editor-only state.
Cook	Dependency closure, validation, and staging for a relocatable build.
Model correction	Asset/render transform that fixes import axis, facing, pivot, or units.
Scene transform	World transform used by gameplay, physics, navigation, gizmos, and scripts.
Fixed step	Constant simulation interval used for deterministic collision-sensitive systems.

This rundown was generated from the current engine, editor, game, player, test, and CMake source structure on 9 August 2026. For member-level APIs and task procedures, use the full engine systems reference and the project tutorials under docs/.

17 - PROJECT PACKAGING

The editor now owns the release workflow. A project can be cooked, compiled, staged, and optionally zipped without leaving the editor. Packaging uses a project-local build directory so it does not disturb the engine's normal development build.

Editor workflow

Stage	What the editor does	Result
1. Prepare	Saves the dirty scene and refreshes Content.	Authored data is current.
2. Cook	Validates dependencies and writes runtime-ready Content.	Cook manifest, player.cfg, and AssetRegistry are present.
3. Configure	Creates an isolated CMake package build for the active project.	Intermediate/Packaging/Build
4. Compile	Builds the player and current project script module.	Selected Release, RelWithDebInfo, or Debug binaries.
5. Stage	Installs the player and cooked project into a relocatable folder.	A runnable project package.
6. Archive	Optionally creates a ZIP beside the staged folder.	A shareable distribution artifact.

Packaging settings

Setting	Purpose
Output folder	Destination for the staged package and optional ZIP. Browse uses the system folder picker.
Configuration	Release for shipping, RelWithDebInfo for profiling, or Debug for development diagnostics.
Static C/C++ runtime	Packages the Microsoft runtime choice with fewer machine-level prerequisites when supported by the build.
Clean previous package	Removes only the package folder and archive owned by this project before staging fresh output.
Create ZIP	Produces <ProjectName>-<Configuration>.zip after staging succeeds.

Open Project > Package Project..., choose an output folder, review the configuration, and select Package Project. The Console reports progress while the work runs in the background.

18 - PACKAGE VERIFICATION AND DELIVERY

A successful compile is not the finish line. The staged folder must remain runnable away from the source tree, and its cooked data must match the scripts and assets that were authored in the project.

Expected output

```
Build/Packages/
WizardTrial/
3DGPlayer.exe
player.cfg
Content/
AssetRegistry.3dgdb
CookManifest.3dgmanifest
game_scripts.dll
WizardTrial-Release.zip
```

The exact executable and library layout follows the player's install rules. The important contract is that the staged player resolves its configuration, registry, cooked Content, scenes, and script module from the package rather than from editor or source directories.

Release checklist

Check	Acceptance condition
Cook	Manifest, player configuration, and registry exist; unresolved required dependencies are zero.
Scripts	The packaged game_scripts library was rebuilt from the active project and loads without errors.
Startup	The configured startup scene or persistent world opens from the staged package.
Isolation	Copy the package to another folder and run it without the editor or source tree.
Gameplay	Test input, camera, physics, AI, animation, HUD, audio, particles, streaming, save/load, and quit flow.
Performance	Use Release or RelWithDebInfo and verify frame time, draw calls, memory, and streaming behavior.

Diagnostics and failure recovery

Symptom	Inspect	Likely action
Cook rejected	Editor Console and validation messages	Repair missing assets, scripts, startup scene, or registry references, then cook again.
Build failed	Intermediate/Packaging/package.log	Resolve compiler/linker errors and rebuild project scripts.
Package starts but content is missing	CookManifest and staged Content	Confirm the asset is reachable from the startup world or explicitly included by cook rules.
Works only beside source tree	Absolute paths and external DLL dependencies	Replace source paths with package-relative resolution and stage required runtime libraries.

Ownership and safety rules

Packaging never writes into authored Content. Clean mode is restricted to the selected project's direct staged folder and ZIP. The package build stays under Intermediate/Packaging, while final artifacts stay under the chosen output folder. These boundaries keep editor data, development builds, and distributable builds separate.